SIMATS ENGINEERING
DATA STRUCTURES ASSIGNMENT

Name	Mohamad Hasan M
Register Number	1924260237B
Subject	Data Structures

A. What is a pointer in C? Explain its relevance (5 marks)

A pointer is a variable that stores the memory address of another variable.

Example:

```
int a = 10;\nint *p = &a;
```

Here, a stores 10, &a gives the address of a, p stores that address, and *p accesses the value at that address.

Relevance of pointers:

1. They provide access to memory addresses.
2. They support dynamic memory allocation.
3. They are used to implement linked lists, stacks, and queues.
4. They allow functions to modify original variables through addresses.
5. They help handle arrays and data structures efficiently.

Conclusion: Pointers are important for memory management and for implementing complex data structures.

B. Design an efficient algorithm for matrix transpose (10 marks)

The transpose of a matrix is obtained by converting its rows into columns and its columns into rows. It is denoted by A^T.

Algorithm:

6. Start.
7. Read the number of rows r and columns c.
8. Read the matrix elements into A.
9. For every element A[i][j], assign T[j][i] = A[i][j].
10. Display the transpose matrix T.
11. Stop.

C program:

```
#include <stdio.h>

int main()
{
    int a[10][10], t[10][10];
    int r, c, i, j;

    printf("Enter rows and columns: ");
    scanf("%d%d", &r, &c);

    printf("Enter matrix elements:\\n");
```


Notewise

```
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            t[j][i] = a[i][j];

    printf("Transpose matrix:\\n");
    for (i = 0; i < c; i++)
    {
        for (j = 0; j < r; j++)
            printf("%d ", t[i][j]);
        printf("\\n");
    }
    return 0;
}
```

Sample input:

```
2 3\n1 2 3\n4 5 6
```

Sample output:

```
Transpose matrix:\n1 4\n2 5\n3 6
```

Time complexity: O(r × c), because each element is processed once.

Space complexity: O(r × c), because a separate matrix stores the transpose.

Conclusion: The algorithm is efficient because it visits each element once.

C. Discuss the use of recursion in file system traversal (5 marks)

Recursion is a technique in which a function calls itself. In file system traversal, it explores directories and their subdirectories.

Working:

12. Start with a main directory.
13. List its files and subdirectories.
14. Process or display each file.
15. When a subdirectory is found, call the same function for that directory.
16. Stop when there are no more subdirectories to explore.

Example directory structure:

```
College\n├── Notes\n|    ├── C.pdf\n|    └── DSA.pdf\n└── Assignments\n     └── Lab.c
```

Advantages:

- Makes directory traversal simple and readable.
- Handles nested folders naturally.
- Avoids writing separate code for every folder level.

Notewise

Conclusion: Recursion is useful because file systems have a hierarchical structure with folders containing other folders.

Notewise